

From simulation to a vertical world model

A technical proposal for the pipeline that turns a mechanistic business simulation into an aggregated corpus and trains a world model that generalizes across an industry.

Fredrik Paulin · June 2026 · technical whitepaper / proposal draft

Abstract

This is the engineering companion to the position paper *Manufacture the data, generalize the model, ground the agent*. That paper argued the case; this one specifies the machinery. We describe a four-stage pipeline — generate, aggregate, train, adapt — that converts a deterministic, mechanistic simulation of an industry archetype into a training corpus and uses it to pretrain a world model of that industry’s operational dynamics. The simulation is parameter-randomized across the industry so that any real member falls inside the training distribution, and it emits uncensored ground truth (true demand, counterfactual lost sales, full cost basis) that no real point-of-sale system records. The corpus is a set of typed, timestamped event trajectories tagged with the scenario parameters and the realized economic outcome. The world model is an autoregressive transformer over a factored event tokenization, with a probabilistic numeric head for the continuous channels, trained on next-event likelihood plus a proper-scoring forecasting loss and a multi-step rollout-consistency term. Adaptation to a specific business is in-context conditioning on its recent logs, optionally with low-rank fine-tuning. We give the corpus schema, the algorithms, the evaluation protocol — built around generalization to held-out parameter regions and recovery of censored demand — and an explicit failure analysis. Nothing here is trained yet; the contribution is the specification and the falsifiable evaluation that would confirm or refute it.

Companion paper

This is one of two technical routes in a three-paper set. The position paper — *Manufacture the data, generalize the model, ground the agent* (position-paper-simulated-world-models.md) — makes the case this document assumes: that an industry poor in per-business data but rich in shared mechanism can have its missing data manufactured from a simulation, that a world model trained on it generalizes across the industry, and that value reaches a specific business through an agentic harness rather than the model alone. It also states the falsifiable thesis the evaluation in §8 is designed to test. Read it for the motivation; read on here for the architecture, corpus, algorithms, and evaluation. For the alternative route — predicting in latent space instead of generating events — see *Predicting in latent space* (whitepaper-joint-embedding-pipeline.md).

1. Scope

This document covers three stages and stops short of the fourth. In scope: the simulation as a data generator (§4), the aggregated corpus and its tokenization (§5), and the world-model architecture, objectives, and training (§6–7). The fourth stage — the agentic harness that consumes the grounded model to act on a real business under human oversight — is referenced where it constrains design but specified elsewhere.

The reader is assumed familiar with sequence modelling, model-based reinforcement learning, and probabilistic forecasting. The worked vertical throughout is a perishable-goods retailer (a bakery), used only as a concrete archetype; nothing in the pipeline is specific to it.

One framing claim governs every design choice below: **the generator is mechanistic, not learned**. Its outputs are constrained by conservation laws — mass balance on inventory, balanced double-entry on money — not by a previous model’s output distribution. This is the structural reason the corpus does not degrade the way recursively generated text does (Shumailov et al., 2024), and it is the line that separates this proposal from “generate synthetic data with an LLM.”

2. Requirements and constraints

The pipeline must satisfy the following. Each is testable; each rules out at least one otherwise-tempting design.

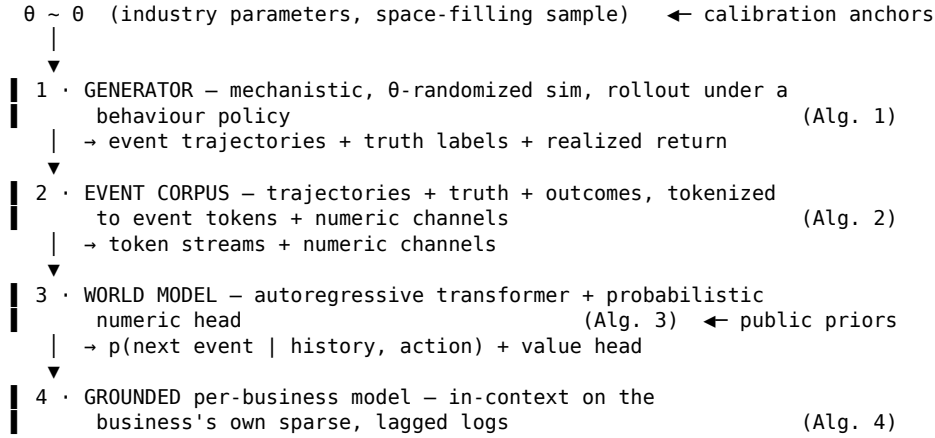
#	Requirement	Consequence
R1	Determinism. A trajectory is a pure function of (parameters, seed).	Reproducible corpora; exact regeneration; defensible ablations.
R2	Mechanistic grounding. Every emitted quantity obeys inventory and accounting conservation.	No internal contradiction; resistance to model collapse.
R3	Industry coverage. The generator is parameterized over the industry’s variation, not tuned to one business.	Domain randomization; a real business is one sample from the training distribution.
R4	Uncensored labels. The corpus records true demand and counterfactual lost sales, not just sales.	A de-censoring signal unavailable from real data.
R5	Schema parity. Simulated events use the same vocabulary the production harness sees.	A model trained on the corpus needs no input remapping to face a real business.
R6	Offline trainability. The corpus is a static, shardable artifact.	Standard large-scale sequence-model training applies.
R7	Transfer-first evaluation. Success is decision quality on held-out businesses, not reconstruction fidelity.	Evaluation splits by parameter region, not by time alone (§8).

R2 and R4 are the requirements that make this worth doing rather than merely possible. R7

is the one most easily violated by accident.

3. System overview

Four stages, top to bottom. Each stage’s output is a durable artifact, so the pipeline is restartable at any boundary.



The remainder of the document specifies each stage and the algorithms that move between them.

4. The simulation as a data generator

4.1 Event model and determinism

The generator advances an append-only event log; state is a projection of that log. Events are typed and timestamped, carrying both `occurredAt` (simulated event time) and, where late arrival is modelled, `receivedAt` (ingestion time), so the corpus can teach a model to reason under the lag and jitter a real business exhibits. A seeded PRNG forked per process makes each rollout a pure function of (θ, seed) — R1.

The generator runs at two fidelities that emit the same event vocabulary (R5). *Aggregate* fidelity treats demand as a probabilistic flow and produces whole simulated quarters in milliseconds, which is what makes a large randomized corpus affordable. *Entity* fidelity resolves the same world into individual customers and staff (agent-based modelling) for trajectories where micro-behaviour matters. The corpus may mix both; the tokenization (§5.2) is identical because the events are.

4.2 The parameter space and how it is sampled

The industry is represented as a parameter vector θ — footfall scale and shape, product mix and margins, supplier lead times and reliability, staffing levels and competence, seasonality and weather sensitivity, opening calendar, shock rates. Θ is the support of plausible businesses in the vertical.

Coverage is the whole game (R3, R7). Independent uniform sampling wastes budget in the interior and under-covers the corners where transfer fails. We propose quasi-random, space-filling designs — Latin hypercube (McKay et al., 1979) or Sobol sequences — over Θ , with density increased near *calibration anchors*: points where real aggregate statistics are available (published seasonality, category demand levels, typical waste rates) and the

generator’s marginals can be matched. This is domain randomization (Tobin et al., 2017) with the anchors keeping the distribution honest rather than merely wide.

4.3 Behaviour policies and action labels

A world model that will inform decisions must see *decisions* and their consequences, across more of the state space than any single good policy visits. Each rollout is therefore driven by a behaviour policy sampled from a portfolio: deterministic baselines (par-stock, moving-average forecast), perturbed variants, and deliberately suboptimal policies that visit stock-outs and gluts. Where available, expert or agent rollouts are added.

The portfolio matters because of the well-known distribution-shift problem in imitation: a model trained only on near-optimal trajectories behaves badly in the off-distribution states its own errors create (Ross et al., 2011). Covering bad states on purpose is how the corpus inoculates the model against compounding error.

4.4 Label richness: the de-censoring advantage

Each trajectory is labelled with what reality hides. A real point-of-sale log is a censored observation of demand — a product that sells out records the sale and forgets the unmet demand — and recovering true demand under censoring is a hard estimation problem with its own literature and benchmarks (FreshRetailNet-50K, 2025). The generator does not estimate true demand; it *generated* it, so each trajectory carries the true demand process, the counterfactual lost sales, and the full per-lot cost basis alongside the observable sales. Training on these labels (R4) gives the world model an internal demand model that is uncensored by construction — the single largest advantage of simulated data over any quantity of real data.

5. The corpus

5.1 Record schema

The unit of training is a trajectory: a scenario parameter vector, an ordered event sequence, the stripped observable view, the hidden truth channel, and the realized economic outcome. Defined as JSON Schema (draft 2020-12) for parity with the production world model:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["theta", "seed", "events", "outcome"],
  "properties": {
    "theta": { "type": "object", "description": "scenario parameters; the industry coordinate" },
    "seed": { "type": "integer" },
    "events": { "type": "array", "items": { "$ref": "#/$defs/event" } },
    "truth": { "type": "array", "description": "hidden truth: true demand, lost sales" },
    "outcome": {
      "type": "object",
      "description": "realized return for value/return modelling",
      "properties": {
        "operatingProfitMinor": { "type": "integer" },
        "serviceLevel": { "type": "number" },
        "wasteFraction": { "type": "number" }
      }
    }
  }
},
"$defs": {
  "event": {
    "type": "object",
```

```

    "required": ["type", "occurredAt", "payload"],
    "properties": {
      "type": { "type": "string" },
      "occurredAt": { "type": "string" },
      "receivedAt": { "type": "string" },
      "payload": { "type": "object" }
    }
  }
}

```

Money is integer minor units throughout; nothing that has to reconcile is ever a float, which also keeps the numeric tokenizer (§5.2) from having to encode rounding noise.

5.2 Tokenization and serialization

Two channels share one timeline.

Symbolic channel. Events are serialized as a stream of factored tokens: an event-type token, then field tokens. Categorical fields draw from a fixed vocabulary; structural tokens delimit records. This “language of operations” is the representation an autoregressive transformer consumes, in the lineage of discrete-token world models (Micheli et al., 2023; Bruce et al., 2024) and sequence-model RL (Chen et al., 2021; Janner et al., 2021), where each input dimension is discretized and predicted in turn.

Numeric channel. Continuous quantities — demand, prices, durations — are encoded by per-series scaling then quantization, the scheme that lets a transformer treat a numeric series as a sequence of tokens (Ansari et al., 2024), or by patching, where a window of points becomes one token (Das et al., 2024). Both event time and ingestion time are encoded as positions so the model can represent lag explicitly.

Two representations of the same trajectory let two model families be compared on identical data (§6.2) without regenerating the corpus.

5.3 Scale and budget

Simulated data is bounded only by compute, which is the inversion of the frontier’s predicament (Villalobos et al., 2024). A rough order of magnitude for one vertical: 10^5 – 10^6 scenarios $\times$ a multi-quarter horizon $\times$ event resolution lands in the low billions of event tokens, comparable to a small-to-mid pretraining run and cheap to regenerate when the schema or the generator improves. Because regeneration is deterministic (R1), the corpus is reproducible rather than archived.

5.4 Hygiene and splits

The collapse risk here is not recursive text but **parameter-distribution bias**: an over-sampled corner of Θ teaches the model a skewed industry. Mitigations: space-filling sampling (§4.2), per- θ -region quotas, and diversity audits over the marginals. Splits are by θ -region, not only by time — held-out regions of the parameter space are the generalization test (R7), and a model that has seen every region has been allowed to memorize the industry rather than learn it.

6. The world model

6.1 What it must represent

The model is a conditional generative model of operational dynamics: the next-event distribution given history and, where the next event is a decision, the action. Formally it approximates $p(o_{t+1} \mid o_{\leq t}, a_t)$ over the event stream, with two heads — a **dynamics head** (generative, the world’s response) and a **value/return head** (predicting the realized outcome) so the same model serves both rollout and decision evaluation. It is a model of how businesses of this kind behave, not a store of any one business’s facts.

6.2 Architecture: options and recommendation

Family	Representative	Strength	Cost
Latent RSSM	PlaNet, Dreamer (Hafner et al., 2019; 2020)	compact latent, efficient imagined rollouts	weaker at heterogeneous symbolic events; less in-context flexible
Autoregressive token transformer	IRIS, Genie, Trajectory/Decision Transformer (Micheli et al., 2023; Bruce et al., 2024; Janner et al., 2021; Chen et al., 2021)	scales; flexible; native in-context conditioning	quadratic context; numeric precision needs care
Time-series foundation head	TimesFM, Chronos, Lag-Llama (Das et al., 2024; Ansari et al., 2024; Rasul et al., 2023)	strong probabilistic forecasting; zero-shot transfer	not a full world model on its own

Recommendation: an autoregressive transformer over the factored event tokenization, with a time-series-foundation-style probabilistic head on the numeric channel.

The deciding factor is the adaptation requirement (§6.4). Per-business grounding wants to condition on a business’s recent history *in context*, without retraining — exactly the regime sequence-model RL operates in (Chen et al., 2021), and what a latent RSSM makes awkward. The numeric head supplies calibrated demand distributions, which the downstream harness needs as distributions, not point estimates.

6.3 Training objectives

The loss is a sum of three terms:

1. **Symbolic next-event likelihood** — cross-entropy over the factored event tokens; the autoregressive backbone.
2. **Numeric probabilistic loss** — a proper scoring rule on the numeric head: quantile (pinball) loss or CRPS (Gneiting & Raftery, 2007), so the model is rewarded for calibrated uncertainty rather than a confident point.

3. **Rollout consistency** — multi-step prediction trained with scheduled sampling and a latent-overshooting-style multi-step objective (Hafner et al., 2019), so error does not compound when the model is unrolled. The value/return head is trained jointly as an auxiliary regression toward the trajectory outcome.

Teacher forcing for the first epochs, then scheduled sampling, is the standard remedy for the gap between training and the autoregressive rollout the model will actually be used in.

6.4 Pretrain, then ground

Pretraining runs on the simulated industry corpus, optionally interleaved with public priors (calendars, weather, holiday and seasonality patterns, published category levels) so the model’s notion of “December” or “a heatwave” is anchored to the real world rather than only to the generator’s.

Grounding a specific business is the adaptation step and the point of the whole exercise: condition the pretrained model on that business’s own recent, sparse, lagged, censored logs — in context, with optional low-rank fine-tuning for stable specifics like its recipe set and supplier list. Because the model was pretrained on uncensored data (R4), conditioning it on a business’s censored sales yields a de-censored demand estimate as a side effect: it has learned, across the industry, what demand looks like behind a stockout. The grounded model exposes two products to the harness — a calibrated forward demand/dynamics distribution, and value estimates for candidate actions.

7. Algorithms

Compact procedures for the four stage transitions. Notation: Θ the parameter support, π a behaviour policy, H the horizon.

Algorithm 1 — Corpus generation (generate $\rightarrow$ corpus).

```
for i in 1..N:
     $\theta \leftarrow \text{sample}(\Theta; \text{space-filling design})$  # §4.2
     $s \leftarrow \text{sample\_seed}()$ 
     $\pi \leftarrow \text{sample}(\text{behaviour\_portfolio})$  # §4.3
    log  $\leftarrow []$ 
    state  $\leftarrow \text{init}(\theta, s)$ 
    for t in 1..H:
        a  $\leftarrow \pi(\text{observe}(\text{state}))$  # decision events
        state, ev, truth  $\leftarrow \text{step}(\text{state}, a, \text{rng}(s, t))$  # mechanistic, conserved (R2)
        log.append(ev); truthlog.append(truth) # uncensored labels (R4)
    outcome  $\leftarrow \text{score}(\text{truthlog})$  # realized return
    emit( $\{\theta, s, \text{events: log, truth: truthlog, outcome}\}$ )
```

Algorithm 2 — Tokenization (corpus $\rightarrow$ tokens).

```
for each trajectory:
    for each event ev in order:
        emit_token(TYPE[ev.type])
        for field, value in ev.payload:
            if categorical(field): emit_token(VOCAB[field][value])
            else: emit_token(quantize(scale(value, field))) # §5.2
        emit_position(ev.occurredAt, ev.receivedAt)
    attach numeric-channel patches for continuous series
```

Algorithm 3 — World-model pretraining (tokens $\rightarrow$ model).

```
for batch in corpus:
    L_sym  $\leftarrow \text{CE}(\text{next\_event\_logits}, \text{targets})$  # §6.3(1)
    L_num  $\leftarrow \text{pinball/CRPS}(\text{numeric\_head}, \text{numeric\_targets})$  # §6.3(2)
    L_roll  $\leftarrow \text{multistep\_consistency}(\text{model}, \text{batch}, \text{k\_steps})$  # §6.3(3), scheduled sampling
    L_val  $\leftarrow \text{MSE}(\text{value\_head}, \text{outcome})$ 
```

```
step(optimizer, L_sym +  $\lambda_1$ ·L_num +  $\lambda_2$ ·L_roll +  $\lambda_3$ ·L_val)
```

Algorithm 4 — Per-business grounding (model $\rightarrow$ grounded model).

```
ctx  $\leftarrow$  recent_business_events (sparse, lagged, censored)           # observation contract
if enough_history: model  $\leftarrow$  LoRA_finetune(model, ctx)           # optional
demand_dist  $\leftarrow$  model.numeric_head(condition=ctx)               # de-censored (R4)
value(a)  $\leftarrow$  model.value_head(condition=ctx, action=a)
return {demand_dist, value}                                           # consumed by the harness
```

8. Evaluation protocol

The position paper’s thesis is falsifiable; this is how.

8.1 Splits. Three: held-out *seeds* within trained θ -regions (in-distribution sanity), held-out θ -regions (generalization across the industry — the decisive sim test, R7), and *real-business* shadow logs (sim-to-real).

8.2 Metrics.

Aspect	Metric
Dynamics	next-event NLL; k-step rollout calibration
Forecasting	CRPS, pinball loss, prediction-interval coverage
Decision quality	regret vs an oracle policy in sim; operating profit net of waste and labour
De-censoring	recovery error of true demand on stockout-censored items (sim ground truth; external check on real censored-demand benchmarks)

8.3 The decisive comparison. On held-out real decisions, does a grounded model beat (a) classical per-business baselines and (b) a generic large model without the world model? The thesis predicts yes, and predicts the margin is largest on stockout-prone, censored items where the uncensored training pays off. If the margin is absent there, the central claim is wrong.

8.4 Ablations. Domain-randomization breadth (narrow vs wide Θ); sim-only vs sim+public priors; in-context grounding vs fine-tune; uncensored labels vs sales-only training. The last isolates the de-censoring advantage directly.

9. Risks and failure modes

Failure analysis at equal weight to the design.

- **Reality gap from unmodeled mechanism.** The generator cannot produce dynamics it does not contain (a new competitor, a viral product). The grounded model will be confidently wrong on these. Mitigation lives downstream: the harness defers to a human at high blast radius. The pipeline’s own mitigation is honest Θ coverage and shadow-mode evaluation that surfaces the gap before deployment.
- **Compounding rollout error.** Autoregressive world models drift over long horizons. Mitigation: rollout-consistency training (§6.3(3)); evaluation explicitly measures k-step calibration, not just one-step NLL.

- **Parameter-distribution bias.** A skewed Θ sample teaches a skewed industry (§5.4). Mitigation: space-filling sampling, per-region quotas, marginal audits, θ -region hold-out.
- **Numeric precision loss in tokenization.** Quantizing money or counts can inject error into quantities that must reconcile. Mitigation: integer minor units; quantization budgets sized per field; reconstruction checks in the corpus build.
- **Residual collapse risk.** If LLM-generated content ever seeds personas or parameters, generator diversity can quietly narrow (Shumailov et al., 2024). Mitigation: the mechanistic core remains the source of dynamics; any learned component is audited for marginal diversity.
- **Overconfident de-censoring.** A strong industry prior on thin local data can produce plausible, wrong demand reconstructions. Mitigation: the numeric head is probabilistic and scored on calibration (§6.3(2)); the harness consumes intervals, not points.
- **Evaluation leakage.** Splitting by time but not by θ lets the model memorize the industry. Mitigation: R7 — θ -region splits are mandatory.

10. Conclusion

The pipeline is an assembly of proven parts pointed at an unproven synthesis. World models (Ha & Schmidhuber, 2018; Hafner et al., 2019, 2020), token-based world models and sequence-model RL (Micheli et al., 2023; Bruce et al., 2024; Chen et al., 2021; Janner et al., 2021), probabilistic time-series foundation models (Das et al., 2024; Ansari et al., 2024), domain randomization (Tobin et al., 2017), and the discipline of proper scoring (Gneiting & Raftery, 2007) are each load-bearing and each already works in its own field. The bet is that a mechanistic, conservation-constrained generator can supply a corpus rich enough — and, uniquely, uncensored enough — to make a world model of an industry’s dynamics that transfers to a real member of it.

The bet is testable to the decimal. Train it on held-out parameter regions, ground it on a real business’s thin logs, and measure whether it beats the baselines on exactly the censored decisions where it should have an unfair advantage. If it does, the data wall is not the long tail’s problem, and never was. If it does not, the failure will be specific, measured, and informative — which is the most a proposal can honestly promise.

References

- Ansari, A. F., et al. (2024). Chronos: Learning the Language of Time Series. *TMLR*. arXiv:2403.07815. <https://arxiv.org/abs/2403.07815>
- Bruce, J., et al. (2024). Genie: Generative Interactive Environments. *ICML 2024*. arXiv:2402.15391. <https://arxiv.org/abs/2402.15391>
- Chen, L., et al. (2021). Decision Transformer: Reinforcement Learning via Sequence Modeling. *NeurIPS 2021*. arXiv:2106.01345. <https://arxiv.org/abs/2106.01345>
- Das, A., Kumar, R., Sen, R., & Zhou, Y. (2024). A decoder-only foundation model for time-series forecasting (TimesFM). *ICML 2024*. arXiv:2310.10688. <https://arxiv.org/abs/2310.10688>
- FreshRetailNet-50K (2025). A Stockout-Annotated Censored Demand Dataset for Latent Demand Recovery and Forecasting in Fresh Retail. arXiv:2505.16319. <https://arxiv.org/abs/2505.16319>

- Gneiting, T., & Raftery, A. E. (2007). Strictly Proper Scoring Rules, Prediction, and Estimation. *Journal of the American Statistical Association*, 102(477), 359–378. <https://doi.org/10.1198/016214506000001437>
- Ha, D., & Schmidhuber, J. (2018). Recurrent World Models Facilitate Policy Evolution. *NeurIPS 2018*. arXiv:1803.10122. <https://arxiv.org/abs/1803.10122>
- Hafner, D., et al. (2019). Learning Latent Dynamics for Planning from Pixels (PlaNet, RSSM). *ICML 2019*. arXiv:1811.04551. <https://arxiv.org/abs/1811.04551>
- Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). Dream to Control: Learning Behaviors by Latent Imagination (Dreamer). *ICLR 2020*. arXiv:1912.01603. <https://arxiv.org/abs/1912.01603>
- Janner, M., Li, Q., & Levine, S. (2021). Offline Reinforcement Learning as One Big Sequence Modeling Problem (Trajectory Transformer). *NeurIPS 2021*. arXiv:2106.02039. <https://arxiv.org/abs/2106.02039>
- McKay, M. D., Beckman, R. J., & Conover, W. J. (1979). A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code (Latin Hypercube Sampling). *Technometrics*, 21(2), 239–245.
- Micheli, V., Alonso, E., & Fleuret, F. (2023). Transformers are Sample-Efficient World Models (IRIS). *ICLR 2023*. arXiv:2209.00588. <https://arxiv.org/abs/2209.00588>
- Rasul, K., et al. (2023). Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting. arXiv:2310.08278. <https://arxiv.org/abs/2310.08278>
- Ross, S., Gordon, G., & Bagnell, D. (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (DAgger). *AISTATS 2011*. arXiv:1011.0686. <https://arxiv.org/abs/1011.0686>
- Shumailov, I., et al. (2024). AI models collapse when trained on recursively generated data. *Nature*, 631, 755–759. <https://www.nature.com/articles/s41586-024-07566-y>
- Tobin, J., et al. (2017). Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. *IEEE/RSJ IROS 2017*. arXiv:1703.06907. <https://arxiv.org/abs/1703.06907>
- Villalobos, P., et al. (2022, rev. 2024). Will we run out of data? Limits of LLM scaling based on human-generated data. *Epoch AI*. arXiv:2211.04325. <https://arxiv.org/abs/2211.04325>